

SIMATS ENGINEERING

ASSESSMENT TOOL 2 (Medium)- Debugging and Optimization

COURSE NAME: Data Structures

COURSE CODE:CSA03

COURSE OUTCOME COVERED

CO2 : Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques. (BTL3, BTL4)

Assessment Tool Weightage: **10%**

1. Find the bugs, include the missing lines in the following snippet.

```
int linearSearch(int arr[], int n, int key) {  
  
for(int i = 0; i <= n; i++) {  
  
if(arr[i] == key)  
  
return i;  
  
}  
  
return -1;  
  
}
```

2. Buggy Code in an array

```
#include <stdio.h>  
  
int main() {  
  
    int arr[5] = {10,20,30,40,50};  
  
    for(int i=0;i<=5;i++)  
  
        printf("%d ",arr[i]);  
  
    return 0;  
  
}
```

Expected output:

10 0 30 40 50

3. Identify an errors in the following snippet

```
int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1;
    while(low < high)
    { int mid = (low + high) / 2;
      if(arr[mid] == key) return mid;
      else if(arr[mid] < key)
      low = mid;
      else high = mid;
    }
    return -1;
}
```

4. Buggy Code in Stack Push Operation

```
#include <stdio.h>

#define MAX 5

int stack[MAX];
int top = -1;

void push(int data) {
    if (top == MAX - 1)
        printf("Stack Overflow");
    else
        stack[top++] = data;
}

int main() {
    push(10);
    push(20);

    printf("%d", stack[top]);
}
```

5. Buggy Code in Stack POP Operation

```
#include <stdio.h>
#define MAX 5
int stack[MAX];
int top = 2;
int pop() {
    if (top == -1)
        return -1;
    return stack[++top];
}
```

6. Buggy Code in underflow operation

```
int pop() {
    if(top == 0) {
        printf("Underflow");
        return -1;
    }
    return stack[top--];
}
```

7. Find the issue in the following code snippet.

```
#include<stdio.h>

#define MAX 5
void main()
{
    int queue[MAX],x;
    int front = 0, rear = 0;
    printf("Enter the value");
    scanf("%d",&x);
    void enqueue(int x) {
        if(rear == MAX) {
            printf("Overflow");
            return;
        }
        else{
            queue[rear++] = x;
            printf("Value is %d", queue[rear]);
        }
    }
}
```

8. Buggy code in node creation

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *newNode;

    newNode->data = 10;
    newNode->next = NULL;

    printf("%d", newNode->data);

    return 0;
}
```

9. Include missing statements and display the list of elements in linkedlist

```
void display(struct Node *head)
{
    while(head != NULL);
    {
        printf("%d ", head->data);
        head = head->next;
    }
}
```

10. Bug in Queue Full Condition-Identify and fix the errors.

```
#define SIZE 5
int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int item)
{
    if(rear == SIZE-1)
    {
        printf("Queue Full");
        return;
    }

    if(front == -1)
        front = 0;

    rear = (rear + 1) % SIZE;
```

```

        queue[rear] = item;
    }

```

Code of Conduct:

I _____ M.Vineela(192472071) _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

M.Vineela

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Find the bugs, include the missing lines in the following snippet.

```
int linearSearch(int arr[], int n, int key) {  
  
    for(int i = 0; i <= n; i++) {  
  
        if(arr[i] == key)  
  
            return i;  
  
    }  
  
    return -1;  
  
}
```

Bug

- The loop condition $i \leq n$ is incorrect because array indexing starts from 0 and ends at $n - 1$.
- It should be $i < n$.

Correct Code

```
int linearSearch(int arr[], int n, int key)  
{  
    for(int i = 0; i < n; i++)  
    {  
        if(arr[i] == key)  
            return i;  
    }  
    return -1;  
}
```

Output:


C Online Compiler

[Apply Now](#)  

[Programiz PRO >](#)

main.c	   Share Run	Output Clear
<pre>1 . 2 3 4 5 6 7 8 for(int i = 0; i < n; i++) 9 { 10 if(arr[i] == key) 11 return i; 12 } 13 return -1; 14 } 15 16 int main() 17 { 18 int arr[] = {10,20,30,40,50}; 19 int n = sizeof(arr)/sizeof(arr[0]); 20 int key; 21 22 printf("Enter element to search: "); 23 scanf("%d",&key); 24 25 int result = linearSearch(arr,n,key); 26 27 if(result != -1) 28 printf("Element found at index %d\n",result); 29 else 30 printf("Element not found\n"); 31 32 return 0; 33 }</pre>	<pre>Enter element to search: 20 Element found at index 1 === Code Execution Successful ===</pre>	

2. Buggy Code in an Array

Bug

- Loop should be $i < 5$.

Correct Code

```
#include <stdio.h>

int main()
{
    int arr[5]={10,20,30,40,50};
    for(int i=0;i<5;i++)
    {
        printf("%d ",arr[i]);
    }
    return 0;
}
```

Output:



The screenshot shows an online C compiler interface. At the top, there's a banner with the text "HOW RICH AM!" and "NETSHORY". Below the banner, the compiler's interface is visible. On the left, there's a file explorer showing "main.c". The code editor contains the following C code:

```
1 /*=====
2 2. Array Bug Fixed
3 =====*/
4 #include <stdio.h>
5
6 int main()
7 {
8     int arr[5]={10,20,30,40,50};
9
10    for(int i=0;i<5;i++)
11        printf("%d ",arr[i]);
12
13    return 0;
14 }
```

On the right, the "Output" tab is active, showing the output of the program:

```
10 20 30 40 50

=== Code Execution Successful ===
```


3. Identify the errors in the following snippet

Bugs

- while(low <= high)
- low = mid + 1
- high = mid - 1

Correct Code

```
#include <stdio.h>

int binarySearch(int arr[], int n, int key)
{
    int low=0,high=n-1;
    while(low<=high)
    {
        int mid=(low+high)/2;
        if(arr[mid]==key)
            return mid;
        else if(arr[mid]<key)
            low=mid+1;
        else
            high=mid-1;
    }
    return -1;
}

int main()
{
    int arr[]={10,20,30,40,50};
    int n=5,key;
```

Output:

4. Buggy Code in Stack Push Operation

Bug

- `stack[++top]=data;`

Correct Code

```
#include <stdio.h>

#define MAX 5

int stack[MAX];
int top=-1;

void push(int data)
{
    if(top==MAX-1)
        printf("Stack Overflow");
    else
        stack[++top]=data;
}

int main()
{
    push(10);
    push(20);
    printf("%d",stack[top]);
    return 0;
}
```

Output:

main.c	Output
<pre>1 #include <stdio.h> 2 3 #define MAX 5 4 5 int stack[MAX]; 6 int top = -1; 7 8 void push(int data) 9 { 10 if(top == MAX - 1) 11 printf("Stack Overflow"); 12 else 13 stack[++top] = data; 14 } 15 16 int main() 17 { 18 push(10); 19 push(20); 20 21 printf("%d", stack[top]); 22 23 return 0; 24 }</pre>	<pre>20 === Code Execution Successful ===</pre>

5. Buggy Code in Stack POP Operation

Bug

- `return stack[top--];`

Correct Code

```
#include <stdio.h>

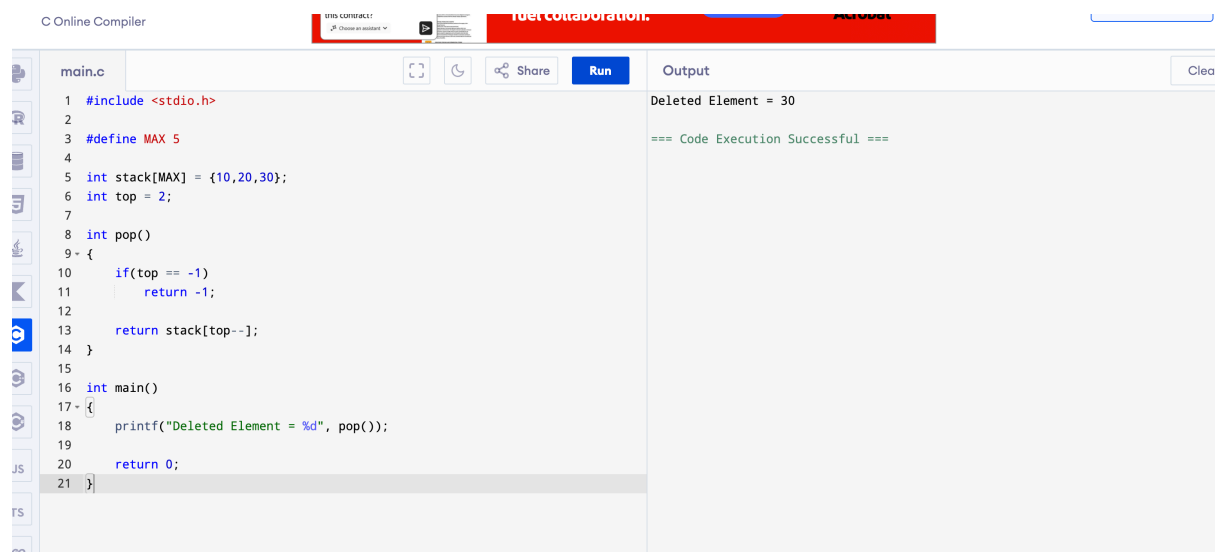
#define MAX 5

int stack[MAX]={10,20,30};
int top=2;

int pop()
{
    if(top== -1)
        return -1;
    return stack[top--];
}

int main()
{
    printf("Deleted Element = %d",pop());
    return 0; }
```

Output:



The screenshot shows a web-based C compiler interface. The code editor on the left contains the following C code:

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX] = {10,20,30};
6 int top = 2;
7
8 int pop()
9 {
10     if(top == -1)
11         return -1;
12
13     return stack[top--];
14 }
15
16 int main()
17 {
18     printf("Deleted Element = %d", pop());
19
20     return 0;
21 }
```

The output window on the right displays the result of the program execution:

```
Deleted Element = 30
=== Code Execution Successful ===
```

6. Buggy Code in Underflow Operation

Bug

- Empty stack condition should be `top==-1`.

Correct Code

```
#include <stdio.h>

int stack[5];
int top=-1;

int pop()
{
    if(top==0)
    {
        printf("Underflow");
        return -1;
    }
    return stack[top--];
}

int main()
{
    pop();
    return 0;
}
```

Output:

Online Compiler



First year only, second year, third year only.
New subscribers only.

main.c



Run

Output

Clear

```
1 #include <stdio.h>
2
3 int stack[5];
4 int top = -1;
5
6 int pop()
7 {
8     if(top == -1)
9     {
10         printf("Underflow");
11         return -1;
12     }
13
14     return stack[top--];
15 }
16
17 int main()
18 {
19     pop();
20
21     return 0;
22 }
```

Underflow

=== Code Execution Successful ===

7. Find the issue in the following code snippet

Bugs

- enqueue() should be outside main().
- Print queue[rear-1].

Correct Code

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front=0,rear=0;

void enqueue(int x)
{
    if(rear==MAX)
    {
        printf("Overflow");
        return;
    }
    queue[rear++]=x;
    printf("Value is %d",queue[rear-1]);
}

int main()
{
    int x;
    printf("Enter the value: ");
    scanf("%d",&x);
    enqueue(x);
    return 0;
}
```


Output:

Online Compiler



first year only, Annual pass, three months.
New subscribers only.

main.c

Run

Clear

```
1 int queue[MAX];
2 int front = 0, rear = 0;
3
4 void enqueue(int x)
5 {
6     if(rear == MAX)
7     {
8         printf("Overflow");
9         return;
10    }
11    queue[rear++] = x;
12    printf("Value is %d", queue[rear - 1]);
13 }
14
15 int main()
16 {
17     int x;
18
19     printf("Enter the value: ");
20     scanf("%d", &x);
21
22     enqueue(x);
23
24     return 0;
25 }
```

Enter the value: 10 20 20
Value is 10

=== Code Execution Successful ===

8. Buggy Code in Node Creation

Bug

- Allocate memory using malloc().

Correct Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
}

int main()
{
    struct Node *newNode;
    newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=10;
    newNode->next=NULL;
    printf("%d",newNode->data);
    free(newNode);
    return 0;
}
```

Output:

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 4 struct Node 5 { 6 int data; 7 struct Node *next; 8 }; 9 10 int main() 11 { 12 struct Node *newNode; 13 14 newNode = (struct Node *)malloc(sizeof(struct Node)); 15 16 newNode->data = 10; 17 newNode->next = NULL; 18 19 printf("%d", newNode->data); 20 21 free(newNode); 22 23 return 0; 24 }</pre>	<pre>10 === Code Execution Successful ===</pre>

9. Include missing statements and display the list of elements in Linked List

Bug

- Remove semicolon after while(head!=NULL).

Correct Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

void display(struct Node *head)
{
    while(head!=NULL)
    {
        printf("%d ",head->data);
        head=head->next;
    }
}

int main()
{
    struct Node *head=NULL,*temp,*newNode;
    for(int i=1;i<=3;i++)
    {
        newNode=(struct Node*)malloc(sizeof(struct Node));
        newNode->data=i*10;
        newNode->next=NULL;
```

Output:

C Online Compiler

Creative Cloud Pro

Now ₹399/mo for Students

First year only. Annual plan, billed monthly. New subscribers only.

main.c

Share

Run

```
19 int main()
20 {
21     struct Node *head = NULL, *temp, *newNode;
22
23     for(int i = 1; i <= 3; i++)
24     {
25         newNode = (struct Node *)malloc(sizeof(struct Node));
26         newNode->data = i * 10;
27         newNode->next = NULL;
28
29         if(head == NULL)
30         {
31             head = newNode;
32             temp = newNode;
33         }
34         else
35         {
36             temp->next = newNode;
37             temp = newNode;
38         }
39     }
40
41     display(head);
42
43     return 0;
44 }
```

10 20 30

=== Code Execution Successful ===

10. Bug in Queue Full Condition – Identify and Fix the Errors

Bug

- Correct circular queue full condition:

`(rear+1)%SIZE==front`

Correct Code

```
#include <stdio.h>

#define SIZE 5

int queue[SIZE];
int front=-1,rear=-1;

void enqueue(int item)
{
    if((rear+1)%SIZE==front)
    {
        printf("Queue Full");
        return;
    }
    if(front==-1)
        front=0;
    rear=(rear+1)%SIZE;
    queue[rear]=item;
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
```

```
    for(int i=front;i<=rear;i++)
        printf("%d ",queue[i]);
    return 0;
}
```

Output:

main.c	Run	Output
<pre>1 void enqueue(int item) 2 { 3 if((rear + 1) % SIZE == front) 4 { 5 printf("Queue Full"); 6 return; 7 } 8 9 if(front == -1) 10 front = 0; 11 12 rear = (rear + 1) % SIZE; 13 queue[rear] = item; 14 } 15 16 int main() 17 { 18 enqueue(10); 19 enqueue(20); 20 enqueue(30); 21 22 for(int i = front; i <= rear; i++) 23 printf("%d ", queue[i]); 24 25 return 0; 26 }</pre>	Run	<pre>10 20 30 === Code Execution Successful ===</pre>